Artifact → GitHub Pages: Skill Guide

Nov 3, 2026 · @p

What it does

The skill takes any Claude artifact you built in chat and puts it live at `https://<your-username>.github.io/<repo>/` in about 5 minutes. Claude drives your own logged-in Chrome to create the repo, upload the files and switch on GitHub Pages, so no GitHub token is ever shared.

It works with React (`.jsx`) and plain HTML artifacts: apps, tools, dashboards, prototypes and landing pages.

Before you start

You need four things in place. If the browser part is missing, the skill still builds the site and hands you a zip with manual steps.

Requirement	Why	How to check
A free GitHub account	The site is hosted in your account	You can open github.com and see your avatar
Logged into GitHub in Chrome	Claude uses your session, never your password	Open github.com/new in Chrome; it shows the form, not a login page
Claude in Chrome extension, connected	Lets Claude click and type in your browser	The extension icon shows connected; Claude Desktop is running
An artifact in the chat	It is what gets deployed	A `.jsx` or `.html` app built earlier in the conversation

The repo is created **public**, because GitHub Pages on free accounts only serves public repos. Anyone can read the code.

Install and share

Install it once from the `artifact-to-github-pages.skill` file:

1. In the chat where the file was shared, click **Save skill** on the file card.

2. Confirm it appears in your skills list (Settings → Capabilities → Skills).
3. Make sure Code Execution and File Creation is on, since the skill runs build scripts.

To give it to a colleague, send them the same `.skill` file and they install it the same way. It always deploys to whichever GitHub account is logged into **their** Chrome, so each person's sites land in their own account.

How to trigger it

Ask in plain words after the artifact is built. Any of these work:

- "Host this on my GitHub."
- "Make this app live on github.io."
- "Push this to my GitHub as a website, repo name `trip-planner`."
- "Deploy it to GitHub Pages, you do it."

Naming the repo is optional; Claude picks a short lowercase name from the app's title if you don't. Keep Chrome open on any tab while it works, and don't click inside the Claude tab group.

What happens, step by step

The skill runs five stages in order. Stages 3 to 5 happen in your Chrome, so you can watch each click.

1. **Build.** A script turns the artifact into `index.html`, `bundle.js` and `README.md`. React loads from a pinned CDN to keep the bundle near 50 KB, and a zip copy is saved as a fallback.
2. **Pick a name.** Claude chooses the repo name and a one-line description, and proofreads it.
3. **Create the repo.** Claude opens github.com/new, reads your username, and creates a public repo.
4. **Upload.** Files are compressed, sent in small chunks, and dropped into GitHub's upload box. Claude checks every file size matches exactly before committing.
5. **Turn on Pages.** Claude sets Settings → Pages to the `main` branch, root folder, then opens the live URL to confirm it renders.

The first Pages build takes 1 to 3 minutes. The final message gives you the live link and anything that works differently once hosted.

What changes once hosted

GitHub Pages serves static files only, so three artifact features behave differently. The skill detects each one and lists it in the final message and the repo README.

Artifact feature	In Claude	On GitHub Pages	Fix if you need it
Saved data (`window.storage`)	Shared across all viewers	Saved in each visitor's own browser	Add a backend such as Firebase or Supabase
AI calls (`api.anthropic.com`)	Work with no API key	Fail; the app should show an error	Route calls through your own server with your own key
`window.claude`, `window.fs`	Available	Undefined	Remove or replace before deploying

Everything else (layout, interactions, local state, images, CDN libraries) works as it did in the chat.

Updating a live site

Change the artifact in chat, then ask Claude to redeploy it to the same repo, for example "push the update to my reclaim repo." Uploading files with the same names overwrites the old ones, and Pages rebuilds on its own in 1 to 2 minutes.

For a one-word fix you can also edit `README.md` or `index.html` directly on github.com with the pencil icon. Don't hand-edit `bundle.js`; it is minified and gets replaced on every redeploy.

Troubleshooting

Most failures come from the browser connection, not GitHub. Find your symptom below.

Symptom	Likely cause	Fix
Claude waits 4 minutes, then says "No result received"	Browser bridge stalled	Restart Claude Desktop, reconnect the Chrome extension, then say "go" to resume

Symptom	Likely cause	Fix
"No tab available"	No tab in Claude's tab group	Claude creates one itself; if it repeats, open any new tab in Chrome
github.io link shows 404	Pages not enabled yet, or first build still running	Wait 2 to 5 minutes; check the repo's Actions tab for a green "pages build and deployment" run
Page loads but is blank	A JavaScript error in the app	Ask Claude to read the console messages on the live page and patch the artifact
GitHub shows a login page	Chrome session expired	Log in yourself, then tell Claude to continue
"Repository already exists"	Name taken in your account	Claude adds a suffix such as `-app`; it never overwrites a repo
Saved data missing for others	Hosted storage is per-browser	Expected; see "What changes once hosted"

If the bridge stalls twice in a row, Claude stops and tells you which stage finished. You can finish by hand with the zip: upload `index.html` and `bundle.js` on the repo page, then set Settings → Pages to `main`, root.

Safety and privacy

The skill acts only inside your own GitHub account and only on the repo it creates for the deploy. It follows these rules every time:

- Never types passwords, tokens or 2FA codes. If GitHub asks you to re-confirm your password, you do that step.
- Never deletes repos or touches other repos.
- Flags anything that looks private before going public, such as real phone numbers, employee IDs, API keys or internal company data, and asks before continuing.

Before deploying a work tool, swap any real colleague data in the demo seed for made-up examples.